PraxisFinance Audit Report

Version 1.0

X: @AlexScherbatyuk

22 Mart 2026

PraxisFinance Audit Report

Alexander Scherbatyuk

22 Mart 2026

Prepared by: X: @AlexScherbatyuk

Lead Auditors:

- Alexander Scherbatyuk

Table of contents

See table

- Table of contents
- About
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
- Protocol Summary
  - Key Components
  - Protocol Flow
  - Roles
- Executive Summary
  - Issues found
- Findings

- Critical
  * [C-1] Inflation attack is possible in YieldToken vault contract, attacker can hijack the vault with low deposit and direct inflation of aToken, preventing new participants to join or for voting participation control.
  * [C-2] The PraxisYTLottery requestDraw uses deprecated v2 version of Chainlink VRF, breaks the lottery contract and leaves users funds staked on the contract.
- High
  * [H-1] The PraxisYTLottery requestDraw check on participant amount less than defined winners amount prevents participants to withdraw if vault is matured and there is no enough participants, leaving funds permanently staked on the contract.
  * [H-2] The PraxisYTLottery fulfillRandomWords function can revert when exceeding 2,500,000 of coordinator maxGasLimit, breaking contracts functionality and staking funds preventing participants to withdraw or claim.
- Low
  * [L-1] Integer division permanently strands YT dust in lottery contract
- Informational
  * [I-1] Missing zero address check in YieldToken constructor
  * [I-2] Missing zero address check in PrincipalToken constructor
  * [I-3] Unprotected renounceOwnership in PraxisVault
  * [I-4] Event emission ordering in PraxisVault::deposit
  * [I-5] Event emission ordering in PraxisVault::withdraw
  * [I-6] Event emission ordering in PraxisVault::redeemYield
  * [I-7] Event emission ordering in PraxisYTLottery::deposit
  * [I-8] Event emission ordering in PraxisYTLottery::withdraw
  * [I-9] Event emission ordering in PraxisYTLottery::requestDraw
  * [I-10] Event emission ordering in PraxisYTLottery::claim

About

Disclaimer

The @AlexScherbatyuk team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

Scope

```
#-- src
|   #-- interfaces
|   |   #-- IAavePool.sol
|   #-- PraxisVault.sol
|   #-- PraxisYTLottery.sol
|   #-- PrincipalToken.sol
|   #-- YieldToken.sol
```

Protocol Summary

PraxisFinance is a decentralized yield farming protocol built on top of Aave that combines yield generation with a gamified lottery mechanism. The protocol enables users to deposit assets into a vault and receive tokenized claims on their principal and yield separately, while offering optional participation in a yield lottery.

Key Components

PraxisVault

- The core vault contract that manages user deposits and yield accumulation. Key features:
- Accepts user deposits and mints Principal Tokens (PT) and Yield Tokens (YT) in a 1:1 ratio
- Deposits assets into Aave's lending pool to generate yield on aTokens
- Tracks total principal and yield separately through internal accounting

- Allows users to redeem their principal and/or yield at maturity
- Enforces a predefined vault maturity date for redemptions

Tokenization Model:

- **Principal Tokens (PT)**: Represent claims on the original deposited amount. Can be redeemed for the exact deposit amount.
- **Yield Tokens (YT)**: Represent claims on the accumulated yield from Aave interest. Can be redeemed for the yield portion at maturity.

PraxisYTLottery - A lottery mechanism that allows Yield Token holders to participate in: - Staking YT tokens into the lottery pool for a chance to win additional rewards - Periodic draws using Chainlink VRF (Verifiable Random Function) to select random winners - Distribution of prizes to winning participants based on random selection

Token Contracts:

- **YieldToken.sol**: ERC-20 token representing yield claims
- **PrincipalToken.sol**: ERC-20 token representing principal claims
- **IAavePool.sol**: Interface to interact with Aave's lending pool for yield generation

Protocol Flow

1. Users deposit assets into PraxisVault and receive equal amounts of PT and YT
2. Vault deposits assets into Aave to generate yield (aToken interest accumulation)
3. YT holders can optionally participate in PraxisYTLottery by staking their YT tokens
4. Lottery conducts periodic draws using Chainlink VRF to select winners
5. Winners receive prizes from the lottery pool
6. At maturity, users redeem PT for principal and YT for accumulated yield

Roles

Role	Description
Vault Depositor	Users who deposit assets into the PraxisVault to receive Principal Tokens (PT) and Yield Tokens (YT). They are entitled to their proportional share of yield and principal at maturity.

Role	Description
Yield Token Holder	Recipients of Yield Tokens (YT) from vault deposits. These tokens represent claims on the accumulated yield generated by the vault's Aave aToken positions.
Principal Token Holder	Recipients of Principal Tokens (PT) from vault deposits. These tokens represent claims on the original deposited principal amount.
Lottery Participant	YT holders who deposit their tokens into the PraxisYTLottery contract for a chance to win additional rewards. Their YT balances are staked for the lottery duration.
Lottery Winner	Randomly selected lottery participants who receive the lottery prize distribution at the end of a draw cycle.
Vault Owner	Contract owner with administrative privileges including renouncing ownership and potential configuration changes.
Protocol/Aave Integration	The external Aave protocol that provides yield accumulation through aToken holdings and interest generation.

Executive Summary

Issues found

Severity	Number of issues found
Critical	2
High	2
Medium	0
Low	1
Info	10
Gas Optimizations	0
Total	15

Findings

Critical

[C-1] Inflation attack is possible in YieldToken vault contract, attacker can hijack the vault with low deposit and direct inflation of aToken, preventing new participants to join or for voting participation control.

Description: The vault protocol allows users to deposit assets and receive Principal Tokens (PT) and Yield Tokens (YT) in proportion to their deposits. The vault lacks enforcement of yield accumulation limits, allowing direct inflation of the underlying aToken balance.

An attacker can exploit this in two ways:

1. **Backrun attack**: Immediately after vault deployment, deposit a minimal amount and directly inflate the vault's aToken balance. This inflates the buy-in cost prohibitively, blocking legitimate participants while the attacker maintains a profitable position.
2. **Dilution attack**: Deposit a significant amount alongside initial participants, then inflate aToken. Existing participants may benefit from the inflation, but new participants face inflated entry requirements due to increased buy-in costs.

```
1          function yieldSurplus() public view returns (uint256) {
2  @>            uint256 assets = totalAssets();
3              uint256 principal = totalPrincipal();
4  @>            return assets > principal ? assets - principal : 0;
5          }
```

Impact:

- Complete hijack of the vault making it unbearable for other participants to pay buy-in and join.
- Manipulation of lottery, voting by controlling amount of participants.

Proof of Concept:

```
1      function test_VaultHijackBackrunPOC() public {
2          uint256 inflation = 1_000_000e6;
3          uint256 initialDeposit = 1 wei;
4
5          address attacker = makeAddr("attacker");
6          address user = makeAddr("user");
7          vm.prank(attacker);
8          usdc.approve(address(vault), type(uint256).max);
9
10         usdc.mint(user, initialDeposit);
11         usdc.mint(attacker, initialDeposit + inflation);
```

```
12
13          console.log("user usdc balanceBefore                  : ", usdc.
                balanceOf(address(user)));
14          console.log("attacker usdc balanceBefore              : ", usdc.
                balanceOf(address(attacker)));
15
16          vm.prank(attacker);
17          vault.deposit(initialDeposit, attacker, 0);
18          console.log("attacker usdc balanceAfter              : ", usdc.
                balanceOf(address(attacker)));
19          console.log("attacker yt balance                     : ", yt.
                balanceOf(address(attacker)));
20          console.log("attacker pt balance                     : ", pt.
                balanceOf(address(attacker)));
21
22          vm.startPrank(attacker);
23          aavePool.addYieldTo(address(vault), inflation);
24          usdc.burn(attacker, inflation);
25          vm.stopPrank();
26
27          uint256 quotedBuyIn = vault.quoteBuyIn(1 wei);
28          console.log("\n");
29          console.log("attacker usdc balanceAfter inflation   : ", usdc.
                balanceOf(address(attacker)));
30          console.log("\n");
31          console.log("vault aToken balance                    : ",
                aToken.balanceOf(address(vault)));
32          console.log("quotedBuyIn for alice to join          : ",
                quotedBuyIn);
33
34          vm.prank(attacker);
35          vault.withdraw(initialDeposit, address(attacker));
36          console.log("attacker usdc balance after withdraw   : ", usdc.
                balanceOf(address(attacker)));
37          assertEq(inflation, quotedBuyIn, "in order to by in need to
                overpay the inflation amount");
38      }
```

```
1       function test_VaultBlockNewParticipantsPOC() public {
2           uint256 inflation = 1_000_000e6;
3           uint256 initialDeposit = 10e6;
4           uint256 secondDeposit = 100_000e6;
5           uint256 thirdDeposit = 1000e6;
6
7           address attacker = makeAddr("attacker");
8           address user = makeAddr("user");
9
10          vm.prank(attacker);
11          usdc.approve(address(vault), type(uint256).max);
12
13          usdc.burn(alice, usdc.balanceOf(address(alice)));
```

```
14          usdc.burn(bob, usdc.balanceOf(address(bob)));
15
16          usdc.mint(attacker, secondDeposit + inflation);
17          usdc.mint(alice, initialDeposit);
18          usdc.mint(bob, thirdDeposit);
19
20          console.log("alice usdc balanceBefore                : ", usdc.
                balanceOf(address(alice)));
21          console.log("attacker usdc balanceBefore             : ", usdc.
                balanceOf(address(attacker)));
22          console.log("bob usdc balanceBefore                  : ", usdc.
                balanceOf(address(bob)));
23          console.log("\n");
24
25          vm.prank(alice);
26          vault.deposit(initialDeposit, alice, 0);
27          console.log("alice usdc balanceAfter                 : ", usdc.
                balanceOf(address(alice)));
28          console.log("alice yt balance                        : ", yt.
                balanceOf(address(alice)));
29          console.log("alice pt balance                        : ", pt.
                balanceOf(address(alice)));
30          console.log("\n");
31
32          vm.prank(attacker);
33          vault.deposit(secondDeposit, attacker, 0);
34          console.log("attacker usdc balanceAfter             : ", usdc.
                balanceOf(address(attacker)));
35          console.log("attacker yt balance                    : ", yt.
                balanceOf(address(attacker)));
36          console.log("attacker pt balance                    : ", pt.
                balanceOf(address(attacker)));
37
38          vm.startPrank(attacker);
39          aavePool.addYieldTo(address(vault), inflation);
40          usdc.burn(attacker, inflation);
41          vm.stopPrank();
42
43          uint256 quotedBuyIn = vault.quoteBuyIn(thirdDeposit);
44          console.log("\n");
45          console.log("attacker usdc balanceAfter inflation   : ", usdc.
                balanceOf(address(attacker)));
46          console.log("\n");
47          console.log("vault aToken balance                    : ",
                aToken.balanceOf(address(vault)));
48          console.log("quotedBuyIn for bob to join            : ",
                quotedBuyIn);
49
50          vm.prank(attacker);
51          vault.withdraw(secondDeposit, address(attacker));
52          console.log("attacker usdc balance after withdraw   : ", usdc.
```

```
            balanceOf(address(attacker)));
53          vm.prank(alice);
54          vault.withdraw(initialDeposit, address(alice));
55          console.log("alice usdc balance after withdraw      : ", usdc.
            balanceOf(address(alice)));
56
57          assertGt(quotedBuyIn, thirdDeposit, "in order to by in need to
            overpay the inflation amount");
58      }
```

Recommended Mitigation: To mitigate direct inflation it is recommended to apply a limit for yield that can be accumulated during the growth period, anything that accrues above that limit should go to the protocol or be distributed to users only after the vault is matured.

Option 1: Take extra yield as donation.

```
1
2   contract PrincipalToken is ERC20 {
3       using SafeERC20 for IERC20;
4
5   +   uint256 public yieldGrowthLimit;
6   +   uint256 public immutable minYieldGrowth;
7   +   error NewYieldGrowthLimitTooLow();
8
9       .
10      .
11      .
12  -   constructor(address _usdc, address _aToken, address _aavePool,
        address _pt, address _yt, uint256 _maturity)
13          Ownable(msg.sender)
14  +   constructor(address _usdc, address _aToken, address _aavePool,
        address _pt, address _yt, uint256 _maturity, uint256 _minYeldGrowth)
15          Ownable(msg.sender)
16      {
17          if (_maturity <= block.timestamp) revert InvalidMaturity();
18
19          usdc = IERC20(_usdc);
20          aToken = IERC20(_aToken);
21          aavePool = IAavePool(_aavePool);
22          pt = PrincipalToken(_pt);
23          yt = YieldToken(_yt);
24          maturity = _maturity;
25  +       minYieldGrowth = _minYieldGrowth;
26  +       yieldGrowthLimit = _minYieldGrowth;
27          .
28          .
29          .
30          function yieldSurplus() public view returns (uint256) {
31              uint256 assets = totalAssets();
32              uint256 principal = totalPrincipal();
33  -           return assets > principal ? assets - principal : 0;
```

```
+            uint256 yield = assets > principal ? assets - principal : 0;
+            return yield > yieldGrowthLimit ? yieldGrowthLimit : yield;
     }
     .
     .
     .

     function rescueTokens(address token, address to, uint256 amount) external onlyOwner {
-        if (token == address(usdc) || token == address(aToken) || token == address(pt) || token == address(yt)) {
+        if (token == address(usdc) || token == address(pt) || token == address(yt)) {
             revert CannotRescueProtectedToken();
         }
         IERC20(token).safeTransfer(to, amount);
         .
         .
         .
         function updateYieldGrowthLimit(uint256 _yieldGrowthLimit) external {
             if (minYieldGrowth < _yieldGrowthLimit) {
                 revert NewYieldGrowthLimitTooLow();
             }
             yieldGrowthLimit = _yieldGrowthLimit
         }
     }
```

Option 2: Distribute extra yield after vault is matured.

```
contract PrincipalToken is ERC20 {
     using SafeERC20 for IERC20;

+    uint256 public yieldGrowthLimit;
     .
     .
     .
-    constructor(address _usdc, address _aToken, address _aavePool, address _pt, address _yt, uint256 _maturity)
         Ownable(msg.sender)
+    constructor(address _usdc, address _aToken, address _aavePool, address _pt, address _yt, uint256 _maturity, uint256 _yieldGrowthLimit)
         Ownable(msg.sender)
     {
         if (_maturity <= block.timestamp) revert InvalidMaturity();

         usdc = IERC20(_usdc);
         aToken = IERC20(_aToken);
```

```
17          aavePool = IAavePool(_aavePool);
18          pt = PrincipalToken(_pt);
19          yt = YieldToken(_yt);
20          maturity = _maturity;
21  +       yieldGrowthLimit = _yieldGrowthLimit;
22          .
23          .
24          .
25          function yieldSurplus() public view returns (uint256) {
26              uint256 assets = totalAssets();
27              uint256 principal = totalPrincipal();
28  -            return assets > principal ? assets - principal : 0;
29  +            uint256 yield = assets > principal ? assets - principal :
        0;
30
31              if(isMatured()){
32                 return yield;
33              }else {
34  +              return yield > yieldGrowthLimit ? yieldGrowthLimit :
        yield;
35              }
36          }
```

[C-2] The PraxisYTLottery requestDraw uses deprecated v2 version of Chainlink VRF, breaks the lottery contract and leaves users funds staked on the contract.

Description: The `requestDraw` function uses the deprecated Chainlink VRF v2 interface to request random words. VRF v2 is no longer supported and has been superseded by v2.5. The v2.5 VRFCoordinator interface differs significantly and requires an additional `extraArgs` parameter that the current implementation does not provide. If the configured VRFCoordinator address corresponds to v2.5 (the recommended version), the function will revert, effectively breaking the lottery draw mechanism. https://docs.chain.link/vrf/v2-5/subscription/get-a-random-number

```
1       function requestDraw() external nonReentrant {
2           if (state != LotteryState.Open) revert LotteryNotOpen();
3           if (!vault.isMatured()) revert VaultNotMatured();
4           if (participants.length < numWinners) {
5               revert NotEnoughParticipants(participants.length,
                    numWinners);
6           }
7
8           state = LotteryState.DrawRequested;
9
10  @>        vrfRequestId = vrfCoordinator.requestRandomWords(
11              keyHash, subscriptionId, REQUEST_CONFIRMATIONS,
                    callbackGasLimit, uint32(numWinners)
12          );
```

```
13
14          emit DrawRequested(vrfRequestId);
15      }
```

Impact: Once the vault matures and `requestDraw` is called, the transaction reverts if the VRFCoordinator address corresponds to v2.5. The lottery enters an unrecoverable state: the vault has matured (preventing withdrawals) and the draw cannot proceed (preventing claims). All participant funds remain locked without recovery mechanism.

Recommended Mitigation: Use v2.5 VRF

1. Use proper imports
2. Use requestRandomWords with correct number of params

```
1  - import {VRFConsumerBaseV2} from "@chainlink/contracts/v0.8/vrf/
     VRFConsumerBaseV2.sol";
2  - import {VRFCoordinatorV2Interface} from "@chainlink/contracts/v0.8/
     interfaces/VRFCoordinatorV2Interface.sol";
3  + import {VRFConsumerBaseV2Plus} from "@chainlink/contracts@1.5.0/src/
     v0.8/vrf/dev/VRFConsumerBaseV2Plus.sol";
4  + import {VRFV2PlusClient} from "@chainlink/contracts@1.5.0/src/v0.8/
     vrf/dev/libraries/VRFV2PlusClient.sol";
5  .
6  .
7  .
8      function requestDraw() external nonReentrant {
9          if (state != LotteryState.Open) revert LotteryNotOpen();
10         if (!vault.isMatured()) revert VaultNotMatured();
11         if (participants.length < numWinners) {
12             revert NotEnoughParticipants(participants.length,
                   numWinners);
13         }
14
15         state = LotteryState.DrawRequested;
16
17 -           vrfRequestId = vrfCoordinator.requestRandomWords(
18 -               keyHash, subscriptionId, REQUEST_CONFIRMATIONS,
     callbackGasLimit, uint32(numWinners)
19 -           );
20 +           vrfRequestId = s_vrfCoordinator.requestRandomWords(
21 +           VRFV2PlusClient.RandomWordsRequest({
22 +               keyHash: keyHash,
23 +               subId: s_subscriptionId,
24 +               requestConfirmations: REQUEST_CONFIRMATIONS,
25 +               callbackGasLimit: callbackGasLimit,
26 +               numWords: uint32(numWinners),
27 +               extraArgs: VRFV2PlusClient._argsToBytes(VRFV2PlusClient.
     ExtraArgsV1({nativePayment: true}))
28 +           })
29 +           );
```

```
30
31          emit DrawRequested(vrfRequestId);
32      }
```

High

[H-1] The PraxisYTLottery requestDraw check on participant amount less than defined winners amount prevents participants to withdraw if vault is matured and there is no enough participants, leaving funds permanently staked on the contract.

Description: The withdraw mechanism enforces maturity checks to prevent withdrawals after vault maturity. Once matured, the only valid action is calling `requestDraw` to finalize the lottery. However, `requestDraw` requires the participant count to exceed the predefined `numWinners` immutable variable. If fewer participants join than expected, `requestDraw` reverts with `NotEnoughParticipants`, creating a deadlock where users cannot withdraw (vault is matured) or proceed (insufficient participants).

```
1       function requestDraw() external nonReentrant {
2           if (state != LotteryState.Open) revert LotteryNotOpen();
3           if (!vault.isMatured()) revert VaultNotMatured();
4
5
6   @>        if (participants.length < numWinners) {
7             revert NotEnoughParticipants(participants.length,
                  numWinners);
8           }
9
10          state = LotteryState.DrawRequested;
11
12          vrfRequestId = vrfCoordinator.requestRandomWords(
13              keyHash, subscriptionId, REQUEST_CONFIRMATIONS,
                  callbackGasLimit, uint32(numWinners)
14          );
15
16          emit DrawRequested(vrfRequestId);
17      }
18      .
19      .
20      .
21      function withdraw(uint256 amount) external nonReentrant {
22          if (state != LotteryState.Open) revert LotteryNotOpen();
23  @>       if (vault.isMatured()) revert VaultAlreadyMatured();
24          if (amount == 0) revert ZeroAmount();
25          if (deposits[msg.sender] == 0) revert NoDeposit();
26
27          uint256 currentDeposit = deposits[msg.sender];
```

```
28          uint256 newBalance = currentDeposit - amount;
29
30          if (newBalance > 0 && newBalance < minDeposit) {
31              revert BelowMinDeposit(newBalance, minDeposit);
32          }
33
34          if (newBalance == 0) {
35              _removeParticipant(msg.sender);
36          }
37
38          deposits[msg.sender] = newBalance;
39          totalDeposits -= amount;
40
41          IERC20(address(yt)).safeTransfer(msg.sender, amount);
42
43          emit Withdrawn(msg.sender, amount);
44      }
```

Impact: If participant count falls below `numWinners`, funds become permanently locked. Users cannot withdraw (vault matured) and cannot claim (draw cannot be requested). No recovery mechanism exists, resulting in total loss of staked funds.

Proof of Concept:

```
1       function test_ParticipatsLessThanWinnersPOC() public {
2           vm.prank(alice);
3           lottery.deposit(200e6);
4           vm.prank(bob);
5           lottery.deposit(200e6);
6
7           vm.warp(block.timestamp + MATURITY + 1);
8
9           vm.expectRevert(abi.encodeWithSignature("NotEnoughParticipants(
                uint256,uint256)", 2, 3));
10          lottery.requestDraw();
11
12          PraxisYTLottery.LotteryState state = lottery.state();
13          assertEq(uint8(state), 0, "lottery state is OPEN");
14          assertEq(vault.isMatured(), true);
15
16          vm.prank(alice);
17          vm.expectRevert(abi.encodeWithSignature("VaultAlreadyMatured()"
                ));
18          lottery.withdraw(200e6);
19
20          vm.prank(bob);
21          vm.expectRevert(abi.encodeWithSignature("VaultAlreadyMatured()"
                ));
22          lottery.withdraw(200e6);
23
24          vm.prank(alice);
```

```
25          vm.expectRevert(abi.encodeWithSignature("LotteryNotFinished()")
                );
26          lottery.claim();
27
28          vm.prank(bob);
29          vm.expectRevert(abi.encodeWithSignature("LotteryNotFinished()")
                );
30          lottery.claim();
31      }
```

Recommended Mitigation: This issue can be addressed with possible solutions:

1. Add additional check for participants amout, if vault has matured, to withdraw function allowing to withdraw if vault is matured but there are less participants than winners (defined amount).

```
function withdraw(uint256 amount) external nonReentrant {
        if (state != LotteryState.Open) revert LotteryNotOpen();
-          if (vault.isMatured()) revert VaultAlreadyMatured();
+       if (vault.isMatured()){
+               if (participants.length >= numWinners) {
+                   revert VaultAlreadyMatured();
+               }
+        }
        if (amount == 0) revert ZeroAmount();
        if (deposits[msg.sender] == 0) revert NoDeposit();

        uint256 currentDeposit = deposits[msg.sender];
        uint256 newBalance = currentDeposit - amount;

        if (newBalance > 0 && newBalance < minDeposit) {
            revert BelowMinDeposit(newBalance, minDeposit);
        }

        if (newBalance == 0) {
            _removeParticipant(msg.sender);
        }

        deposits[msg.sender] = newBalance;
        totalDeposits -= amount;

        IERC20(address(yt)).safeTransfer(msg.sender, amount);

        emit Withdrawn(msg.sender, amount);
    }
```

2. Make `numWinners`storage variable, inherit ownable extation, and add `setNumWinners` function for adming to be able adjust `numWinners` accordingly.

```
-    contract PraxisYTLottery is VRFConsumerBaseV2, ReentrancyGuard {
```

```
+    contract PraxisYTLottery is VRFConsumerBaseV2, ReentrancyGuard,
    Ownable {
        .
        .
        .
-         uint256 public immutable numWinners;
+         uint256 public numWinners;
         .
         .
         .
+         function setNumWinners(uint256 num) public onlyOwner {
+             numWinners = num;
+         }
         .
         .
         .
     }
```

[H-2] The PraxisYTLottery fulfillRandomWords function can revert when exceeding 2,500,000 of coordinator maxGasLimit, breaking contracts functionality and staking funds preventing participants to withdraw or claim.

Description: The `fulfillRandomWords` callback function contains nested loops for winner selection that consume gas quadratically with participant count. The implementation can process approximately 900 participants before exceeding the 2,500,000 gas limit imposed by the Chainlink VRF coordinator. Exceeding this limit causes the callback to revert.

Impact: The lottery contract allows only a single `requestDraw` call per cycle and does not permit retry if the callback fails. If participant count exceeds the gas limit, the VRF callback reverts, permanently locking the lottery in `DrawRequested` state. Users cannot withdraw (vault matured) or claim (draw unresolved), resulting in total fund lockup.

Proof of Concept:

```
function test_fulfillRandomWordsOutOfGas() public {
        uint256 maxGasLimit = 2500000;
        address[] memory users = new address[](950);
        for (uint256 i = 0; i < users.length; i++) {
            users[i] = address(uint160(100 + i));
        }

        uint256 validParticipants = 0;
        for (uint256 i = 0; i < users.length; i++) {
            if (users[i] < address(10)) continue;
            validParticipants++;
            usdc.mint(users[i], 10000e6);
            vm.startPrank(users[i]);
```

```
            usdc.approve(address(vault), type(uint256).max);
            vault.deposit(1000e6, users[i], 0);
            yt.approve(address(lottery), type(uint256).max);
            lottery.deposit(200e6);
            vm.stopPrank();
        }
        vm.warp(block.timestamp + MATURITY + 1);

        lottery.requestDraw();
        uint256 requestId = lottery.vrfRequestId();

        uint256[] memory randomWords = new uint256[](NUM_WINNERS);
        randomWords[0] = 0;
        randomWords[1] = 850;
        randomWords[2] = 950;

        uint256 gasBeforeFulfill = gasleft();

        vrfCoordinator.fulfillRandomWords(requestId, randomWords);
        uint256 gasUsed = gasBeforeFulfill - gasleft();

        console.log("gasUsed: ", gasUsed);
        assertGt(maxGasLimit, gasUsed, "The gas used must be within `
            maxGasLimit` of Chainlink coordinator");
    }
```

results for 950 participants:

```
Logs:
  gasUsed:  24 98 241
```

results for 950 participants:

```
Logs:
  gasUsed:  25 22 752
```

Recommended Mitigation: Reduce the number of inner loops within `fulfillRandomWords`, store requested words during `fulfillRandomWords` call and calculate winners in separate transaction with mitigation allowing to process in case of high demand.

Low

[L-1] Integer division permanently strands YT dust in lottery contract

Description `prizePerWinner = totalDeposits / numWinners` truncates the remainder (`totalDeposits % numWinners`, up to `numWinners - 1` tokens) into the lottery contract;

there is no rescue, sweep, or residual-claim function, and `PraxisVault.rescueTokens` explicitly blocks YT, making the dust unrecoverable.

Impact: Contract accomulate dus from integer division that cannot be withdrawn or trasfered.

Proof of Concept:

```
function test_IntegerMathDustPOC() public {
        vm.prank(alice);
        lottery.deposit(200e6);
        vm.prank(bob);
        lottery.deposit(190e6);
        vm.prank(charlie);
        lottery.deposit(200e6);
        vm.prank(dave);
        lottery.deposit(180e6);

        vm.warp(block.timestamp + MATURITY + 1);

        lottery.requestDraw();
        uint256 requestId = lottery.vrfRequestId();

        uint256[] memory randomWords = new uint256[](NUM_WINNERS);
        randomWords[0] = 0;
        randomWords[1] = 850;
        randomWords[2] = 950;

        vrfCoordinator.fulfillRandomWords(requestId, randomWords);

        uint256 remainder = lottery.totalDeposits() - (lottery.
            prizePerWinner() * NUM_WINNERS);
        console.log("totalDeposits: ", lottery.totalDeposits());
        console.log("prizePerWinner: ", lottery.prizePerWinner());
        console.log("remainder: ", remainder);

        assertGt(remainder,0,"Integer math dust")
    }
```

Recommended Mitigation: Transfer dust to treasury address.

```
+  address treasury;
- constructor( address _vault,  address _vrfCoordinator, bytes32
    _keyHash, uint64 _subscriptionId, uint32 _callbackGasLimit, uint256
    _numWinners, uint256 _minDeposit) VRFConsumerBaseV2(_vrfCoordinator)
     {
+ constructor( address _vault,  address _vrfCoordinator, bytes32
    _keyHash, uint64 _subscriptionId, uint32 _callbackGasLimit, uint256
    _numWinners, uint256 _minDeposit, address _treasury)
    VRFConsumerBaseV2(_vrfCoordinator) {
    treasury = _treasury

```

```
6     .
7     .
8     .
9     prizePerWinner = totalDeposits / numWinners;
10    uint256 remainder = totalDeposits % numWinners;
11  + if (remainder > 0) {
12  +     // assign to last winner or a designated fee recipient
13  +     IERC20(address(yt)).safeTransfer(treasury, remainder);
14  + }
```

Informational

[I-1] Missing zero address check in `YieldToken` constructor

Description: The constructor assigns the vault address parameter without validating against zero address.

```
1  @>  vault = _vault;
```

Impact: An accidental zero address assignment would cause contract redeployment or protocol launch failures.

Recommended Mitigation: Add check for address(0) and revert preventing the deployment of such contract.

```
1  -     vault = _vault;
2  +     if (_vault == address(0)) revert ZeroAddress();
```

[I-2] Missing zero address check in `PrincipalToken` constructor

Description: The constructor assigns the vault address parameter without validating against zero address.

```
1  @>  vault = _vault;
```

Impact: An accidental zero address assignment would cause contract redeployment or protocol launch failures.

Recommended Mitigation:

Add check for address(0) and revert preventing the deployment of such contract.

```
1  -     vault = _vault;
2  +     if (_vault == address(0)) revert ZeroAddress();
```

[I-3] Unprotected `renounceOwnership` in `PraxisVault`

Description: The OpenZeppelin Ownable implementation includes an unprotected `renounceOwnership` function that allows the owner to permanently transfer ownership to address(0).

Impact:

- Permanent loss of access to all owner-restricted functions
- Critical administrative operations become unavailable

Recommended Mitigation: Override the renounceOwnership function to prevent it's use.

```
+    function renounceOwnership() public override onlyOwner {}
```

[I-4] Event emission ordering in `PraxisVault::deposit`

Description: The Deposit event is emitted after external token transfer calls, violating the convention of emitting events before external interactions. This can cause indexers and dApps to process state changes in incorrect order.

```
    function deposit(uint256 principalAmount, address receiver, uint256
        maxBuyIn) external nonReentrant whenNotPaused {
        .
        .
        .
        pt.mint(receiver, principalAmount);
        yt.mint(receiver, principalAmount);
@>      emit Deposit(principalAmount, buyIn, receiver);
    }
```

Impact: Indexers and dApps may incorrectly process state changes due to events being logged after external interactions.

Recommended Mitigation:

```
    function deposit(uint256 principalAmount, address receiver, uint256
        maxBuyIn) external nonReentrant whenNotPaused {
        .
        .
        .
+        emit Deposit(principalAmount, buyIn, receiver);
        pt.mint(receiver, principalAmount);
        yt.mint(receiver, principalAmount);
-        emit Deposit(principalAmount, buyIn, receiver);
    }
```

[I-5] Event emission ordering in `PraxisVault::withdraw`

Description: The Withdraw event is emitted after external token transfer calls, violating the convention of emitting events before external interactions. This can cause indexers and dApps to process state changes in incorrect order.

```
    function withdraw(uint256 amount, address receiver) external
       nonReentrant whenNotPaused {
        if (amount == 0) revert ZeroAmount();

        uint256 yieldPayout = 0;
        .
        .
        .
        pt.burn(msg.sender, amount);
        yt.burn(msg.sender, amount);
        .
        .
        .
        uint256 totalPayout = amount + yieldPayout;
        aavePool.withdraw(address(usdc), totalPayout, receiver);

@>        emit Withdraw(amount, yieldPayout, receiver);
    }
```

Impact: Indexers and dApps may incorrectly process state changes due to events being logged after external interactions.

Recommended Mitigation:

```
    function withdraw(uint256 amount, address receiver) external
       nonReentrant whenNotPaused {
        if (amount == 0) revert ZeroAmount();

        uint256 yieldPayout = 0;
+        emit Withdraw(amount, yieldPayout, receiver);
        .
        .
        .
        pt.burn(msg.sender, amount);
        yt.burn(msg.sender, amount);
        .
        .
        .
        uint256 totalPayout = amount + yieldPayout;
        aavePool.withdraw(address(usdc), totalPayout, receiver);

-        emit Withdraw(amount, yieldPayout, receiver);
    }
```

[I-6] Event emission ordering in `PraxisVault::redeemYield`

Description: The RedeemYield event is emitted after external token transfer calls, violating the convention of emitting events before external interactions. This can cause indexers and dApps to process state changes in incorrect order.

```
1      function redeemYield(uint256 ytAmount, address receiver) external
          nonReentrant whenNotPaused {
2          .
3          .
4          .
5          yt.burn(msg.sender, ytAmount);
6
7          if (payout > 0) {
8              aavePool.withdraw(address(usdc), payout, receiver);
9          }
10
11 @>         emit RedeemYield(ytAmount, payout, receiver);
12     }
```

Impact: Indexers and dApps may incorrectly process state changes due to events being logged after external interactions.

Recommended Mitigation:

```
1      function redeemYield(uint256 ytAmount, address receiver) external
          nonReentrant whenNotPaused {
2          .
3          .
4          .
5  +       emit RedeemYield(ytAmount, payout, receiver);
6
7          yt.burn(msg.sender, ytAmount);
8
9          if (payout > 0) {
10             aavePool.withdraw(address(usdc), payout, receiver);
11         }
12
13 -        emit RedeemYield(ytAmount, payout, receiver);
14     }
```

[I-7] Event emission ordering in `PraxisYTLottery::deposit`

Description: The Deposited event is emitted after external token transfer calls, violating the convention of emitting events before external interactions. This can cause indexers and dApps to process state changes in incorrect order.

```
    function deposit(uint256 amount) external nonReentrant {
        .
        .
        .
        IERC20(address(yt)).safeTransferFrom(msg.sender, address(this),
            amount);

@>        emit Deposited(msg.sender, amount);
    }
```

Impact: Indexers and dApps may incorrectly process state changes due to events being logged after external interactions.

Recommended Mitigation:

```
         function deposit(uint256 amount) external nonReentrant {
        .
        .
        .
+        emit Deposited(msg.sender, amount);
        IERC20(address(yt)).safeTransferFrom(msg.sender, address(this),
            amount);

-        emit Deposited(msg.sender, amount);
    }
```

[I-8] Event emission ordering in `PraxisYTLottery::withdraw`

Description: The Withdrawn event is emitted after external token transfer calls, violating the convention of emitting events before external interactions. This can cause indexers and dApps to process state changes in incorrect order.

```
    function withdraw(uint256 amount) external nonReentrant {
            .
            .
            .

            IERC20(address(yt)).safeTransfer(msg.sender, amount);

@>          emit Withdrawn(msg.sender, amount);
        }
```

Impact: Indexers and dApps may incorrectly process state changes due to events being logged after external interactions.

Recommended Mitigation:

```
    function withdraw(uint256 amount) external nonReentrant {
```

```
            .
            .
            .
+           emit Withdrawn(msg.sender, amount);
            IERC20(address(yt)).safeTransfer(msg.sender, amount);

-           emit Withdrawn(msg.sender, amount);
        }
```

[I-9] Event emission ordering in `PraxisYTLottery::requestDraw`

Description: The DrawRequested event is emitted after the external call to request random words from the Chainlink coordinator, violating the convention of emitting events before external interactions. This can cause indexers and dApps to process state changes in incorrect order.

```
    function requestDraw() external nonReentrant {
     .
     .
     .
        vrfRequestId = vrfCoordinator.requestRandomWords(
            keyHash, subscriptionId, REQUEST_CONFIRMATIONS,
                callbackGasLimit, uint32(numWinners)
        );

@>        emit DrawRequested(vrfRequestId);
    }
```

Impact: Indexers and dApps may incorrectly process state changes due to events being logged after external interactions.

Recommended Mitigation:

```
    function requestDraw() external nonReentrant {
     .
     .
     .
+        emit DrawRequested(vrfRequestId);
        vrfRequestId = vrfCoordinator.requestRandomWords(
            keyHash, subscriptionId, REQUEST_CONFIRMATIONS,
                callbackGasLimit, uint32(numWinners)
        );

-        emit DrawRequested(vrfRequestId);
    }
```

[I-10] Event emission ordering in `PraxisYTLottery::claim`

Description: The PrizeClaimed event is emitted after external token transfer calls, violating the convention of emitting events before external interactions. This can cause indexers and dApps to process state changes in incorrect order.

```
    function claim() external nonReentrant {
         .
         .
         .
         IERC20(address(yt)).safeTransfer(msg.sender, prizePerWinner);

@>         emit PrizeClaimed(msg.sender, prizePerWinner);
     }
```

Impact: Indexers and dApps may incorrectly process state changes due to events being logged after external interactions.

Recommended Mitigation:

```
    function claim() external nonReentrant {
         .
         .
         .
+         emit PrizeClaimed(msg.sender, prizePerWinner);
         IERC20(address(yt)).safeTransfer(msg.sender, prizePerWinner);

-         emit PrizeClaimed(msg.sender, prizePerWinner);
     }
```